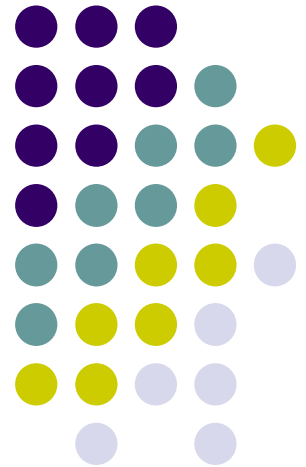
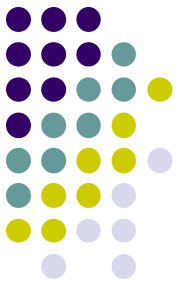


# Database Systems and Data-driven Applications

## Query Optimisation





# Nested Loop Join

- Assume joining relations R and S,

$$R \bowtie_{R.a=S.b} S$$

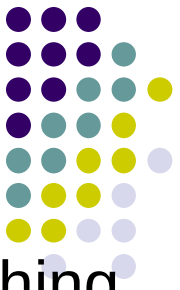
For each tuple r in R do

    for each tuple s in S do

        if  $r_i == s_j$  then add  $\langle r, s \rangle$  to result

- Three algorithms
  - Simple Nested Loop Join
  - Page Oriented Nested Loop Join
  - Block Oriented Nested Loop Join

# Simple Nested Loops join



- For each tuple of R, get all tuple of S, and write out matching pairs of tuples  $\langle r, s \rangle$ , where r is in R-page and S is in S-page.
- Assume M pages in R, N pages in S and  $P_R$  tuples per page in R.

$$Cost = M + M * P_R * N$$

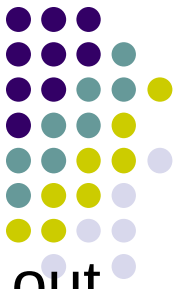
- Minimum buffer needed is 3 pages.
- Selecting the smaller relation as outer may gives a low cost.
- $SP \bowtie_{SP.sno=S.sno} S$ , SP as outer

$$Cost = 1000 + 1000 * 100 * 100 = 1000 + 10,000,000 = 10,001,000 \text{ IOs}$$

- $S \bowtie_{SP.sno=S.sno} SP$ , S as outer

$$Cost = 100 + 100 * 10 * 1000 = 100 + 1,000,000 = 1,000,100 \text{ IOs}$$

# Page Oriented Nested Loops join



- For each page read from R, get all pages of S, and write out matching pairs of tuples  $\langle r, s \rangle$ , where  $r$  is in R-page and  $S$  is in S-page.
- Assume  $M$  pages in R and  $N$  pages in S.

$$Cost = M + M * N$$

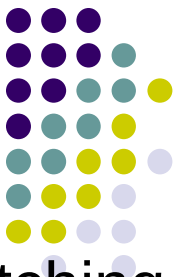
- Minimum buffer needed is 3 pages.
- Selecting the smaller relation as outer gives the lowest cost.

- $SP \bowtie_{SP.sno=S.sno} S$ , SP as outer

$$Cost = 1000 + 1000 * 100 = 1000 + 100,000 = 101,000 \text{ IOs}$$

- $S \bowtie_{SP.sno=S.sno} SP$ , S as outer

$$Cost = 100 + 100 * 1000 = 100 + 100,000 = 100,100 \text{ IOs}$$



# Block Oriented Nested Loops join

- For each block of R, get all pages of S, and write out matching pairs of tuples  $\langle r, s \rangle$ , where  $r$  is in R-page and  $S$  is in S-page.
- Assume  $M$  pages in R,  $N$  pages in S and buffer size is  $B$ .

$$Cost = M + \left\lceil \frac{M}{B-2} \right\rceil * N$$

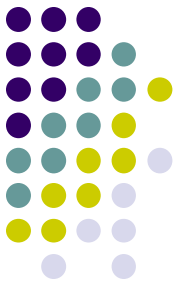
- Minimum buffer needed is 3 pages. If buffer size is 3 the algorithm is same as page oriented nested loop join.
- Selecting the smaller relation as outer gives the lowest cost.
- $SP \bowtie_{SP.sno=S.sno} S$ ,  $SP$  as outer. (Assume  $B=12$ )

$$Cost = 1000 + \left\lceil \frac{1000}{12-2} \right\rceil * 100 = 1000 + 10,000 = 11,000 \text{ IOs}$$

- $S \bowtie_{SP.sno=S.sno} SP$ ,  $S$  as outer. (Assume  $B=12$ )

$$Cost = 100 + \left\lceil \frac{100}{12-2} \right\rceil * 1000 = 100 + 10,000 = 10,100 \text{ IOs}$$

# Index Nested Loops Join

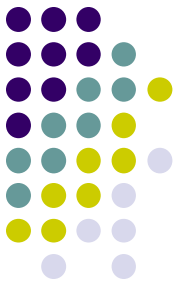


For each tuple  $r$  in  $R$  do  
    for each tuple  $s$  in  $S$  where  $r_i == s_j$  do  
        add  $\langle r, s \rangle$  to result

- If there is an index on the join column of one relation (say  $S$ ), can make it the inner and use the index.
- For each tuple in  $R$  find matching tuples of  $S$  using the index.
- Assume  $M$  pages in  $R$ ,  $N$  pages in  $S$  and  $P_R$  tuples per page in  $R$ .

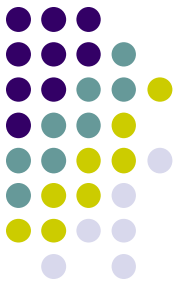
$$\text{Cost} = M + M * P_R * (\text{Cost of getting matching } S \text{ tuples})$$

# Index Nested Loop Join ...



- For each R tuple, cost of probing S index:
  - About 1.2 for hash index,
  - 2-4 for B+ tree.
- Cost of finding S tuples (assuming  $\langle \text{key}, \text{rid} \rangle$  or  $\langle \text{key}, \text{rid list} \rangle$  for index entries) depends on
  - Type of index (B+ / Hash)
  - Clustering.
  - Number of matching S tuples for each R tuple

# Example of Index Nested Loops



- $SP \bowtie_{SP.sno=S.sno} S$ , SP as outer.
- sno is the primary key of the inner relation. One tuple in outer exactly matches one tuple in inner.
- Since one tuple in inner matches the join condition clustering has no effect on the cost.

$$\text{Cost} = 1000 + 1000 * 100 * (\text{Cost of g.m.t.})$$

- Clustered or un-clustered Hash index on  $S\langle sno \rangle$  (S as inner):

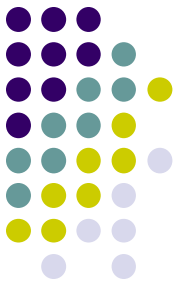
$$\text{Cost of g.m.t.} = 1.2 + 1 \text{ IO}$$

- Clustered or un-clustered B+ index on  $S\langle sno \rangle$  (S as inner):

$$\text{Cost of g.m.t.} = h + 1 \text{ IO}$$



# Example of Index Nested Loops



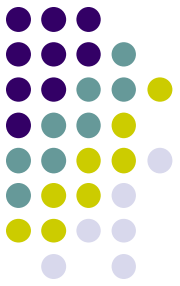
- $S \bowtie_{SP.sno=S.sno} SP$ , S as outer.
- sno is NOT a candidate key in the inner relation. One tuple in outer matches many tuple in inner.
  - 100 SP tuples (1 page) matches one S tuple

$$\text{Cost} = 100 + 100 * 10 * (\text{Cost of g.m.t.})$$

- Clustering has an effect on the cost.
- Clustered Hash index on  $SP\langle sno \rangle$  (SP as inner):

$$\text{Cost of g.m.t.} = 1.2 + RF * \text{Data Pages} = 1.2 + \frac{0.1}{100} * 1000 = 1.2 + 1 = 2.2 \text{ IO}$$

# Example of Index Nested Loops



- Un-clustered Hash index on SP<sno> (SP as inner):

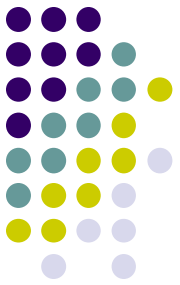
$$\text{Cost of g.m.t.} = 1.2 + RF * \text{Data Record} = 1.2 + \frac{0.1}{100} * 100000 = 1.2 + 100 = 101.2 \text{ IO}$$

- Clustered B+ index on SP<sno> (SP as inner):

$$\text{Cost of g.m.t.} = h + RF * (\text{Leaf Pages} + \text{Data Pages}) = 3 + \frac{0.1}{100} * (334 + 1000) = 3 + 1 + 1 = 5 \text{ IOs}$$

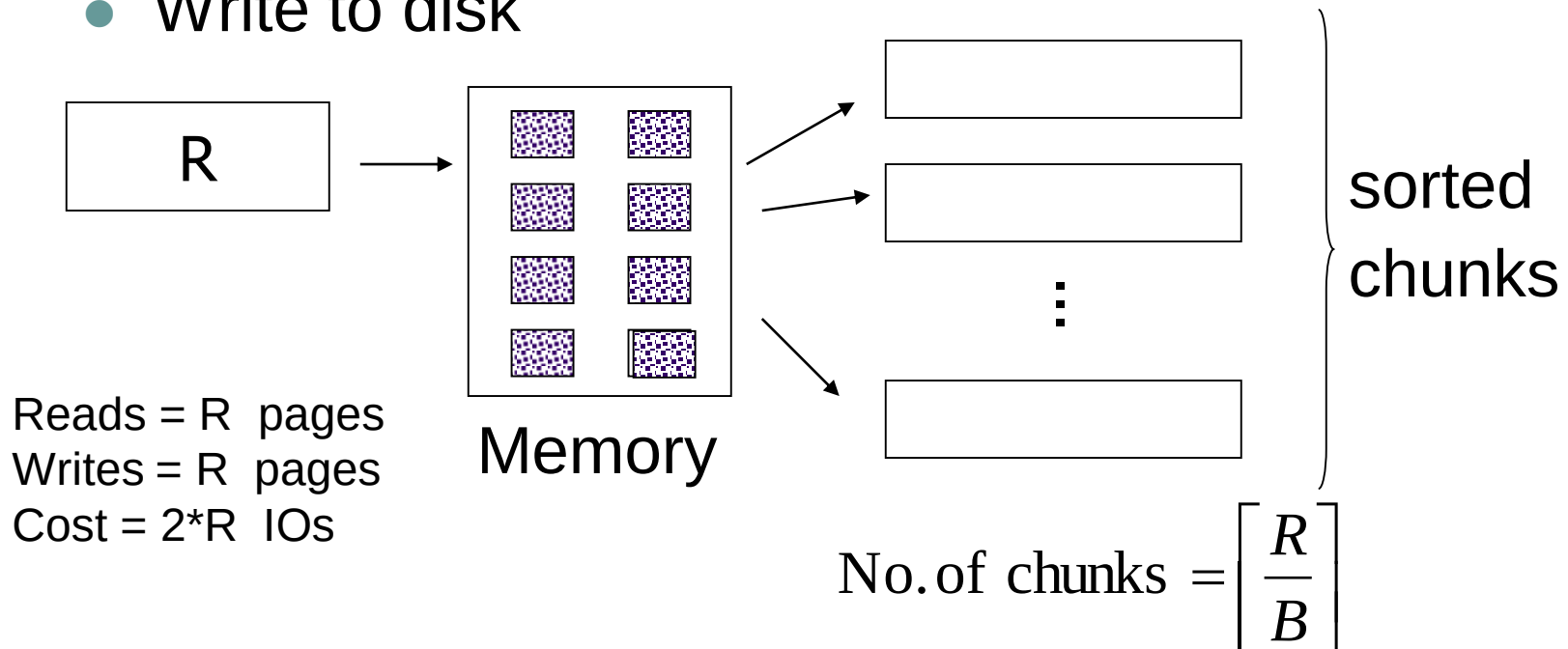
- Un-clustered B+ index on SP<sno> (SP as inner):

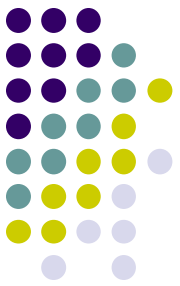
$$\text{Cost of g.m.t.} = h + RF * (\text{Leaf Pages} + \text{Data Record}) = 3 + \frac{0.1}{100} * (334 + 100000) = 3 + 1 + 100 = 104 \text{ IO}$$



# External Merge Sort (with two passes)

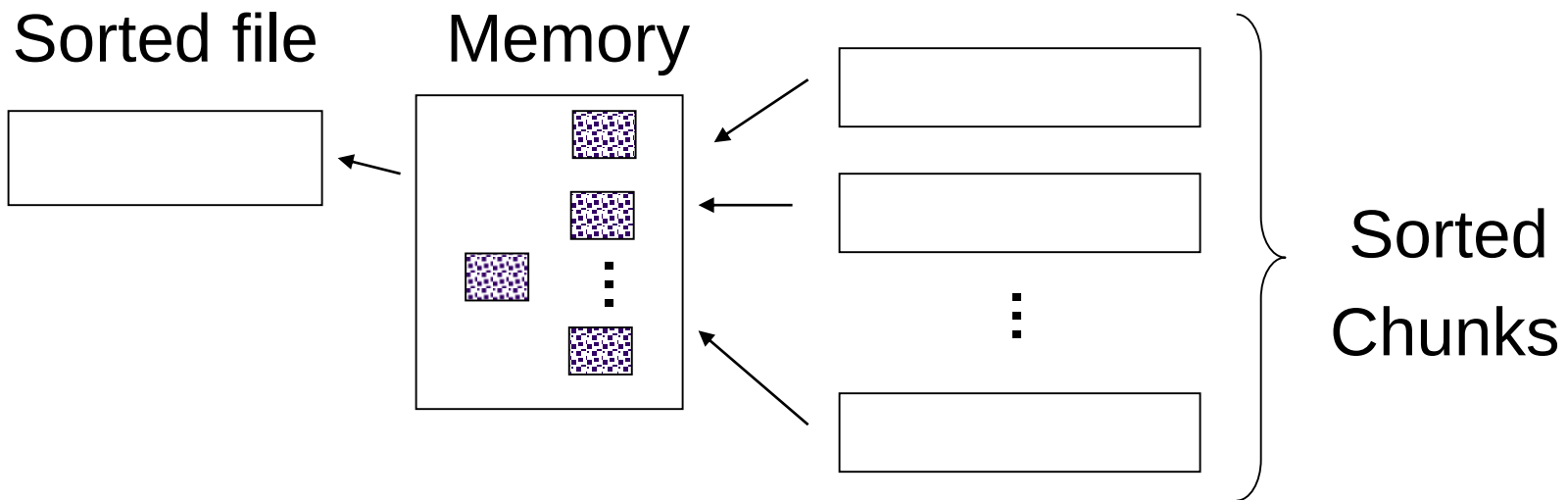
- Sorting take place in two passes
- Pass – 0
  - Read the relation as chunks of several pages
  - Sort in memory
  - Write to disk





# External Merge Sort (with two passes)....

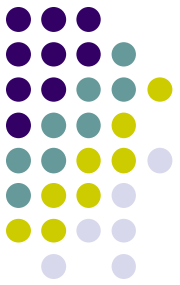
- Pass – 1
  - Read all chunks + merge + write out



Reads =  $R$  pages  
Writes =  $R$  pages  
Cost =  $2 \cdot R$  IOs

Buffer size  $\geq (\text{No. of chunks}) + 1$

$$\text{Min Buffer Size} = \lceil \sqrt{R} \rceil + 1$$



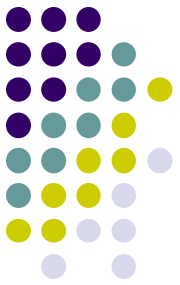
# Sort-Merge Join ( $R \bowtie_{R.a=S.b} S$ )

- Sort R and S on the join column, then scan them to do a “merge” (on join col.), and output result tuples.
  - R is scanned once;
  - Each S group (matching S tuples of an R tuple) is scanned once per matching R tuple.
  - Multiple scans of an S group are likely to find needed pages in buffer.

$$\text{Cost} = \text{Sort R} + \text{Sort S} + \text{Merge R \& S}$$

- Best Case : For each R page it is enough to read one S page for merging.

$$\text{Cost} = 4 * M + 4 * N + (M + N)$$



# Sort-Merge Join .....

- Worst Case (Very unlikely)
  - Every tuple in R matches all tuples in S.
  - All N blocks of S read for each block of R during merge (e.g., join attr has just one value).

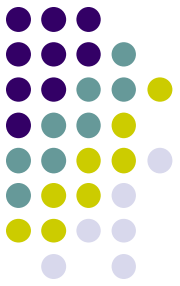
$$\text{Merge Cost} = M + M * N$$

- Cost of  $S \bowtie_{SP.sno=S.sno} SP$ .

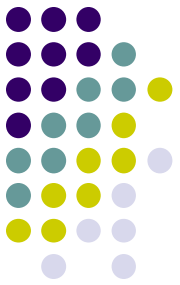
$$\text{Cost} = 4 * 100 + 4 * 1000 + (100 + 1000) = 400 + 4000 + 1100 = 5500 \text{ IOs}$$

$$\text{Min Buffer} = \left\lceil \sqrt{1000} \right\rceil + 1 = 32 + 1 = 33 \text{ Pages}$$

# Aggregate Operations



- Without grouping:
  - In general, requires scanning the relation.
    - Scan relation to accumulate values (e.g. `select sum(salary) from emp`)
  - Given index whose search key includes all attributes in the SELECT or WHERE clauses, can do index-only scan.
    - Using index on `<dept, salary>` to process the query:  
`select avg(salary) from emp where dept = 'D1'`

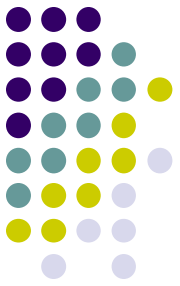


# Aggregate Operations

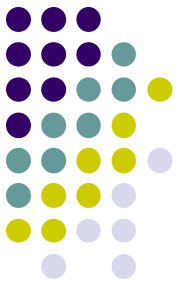
- With grouping:
  - Sort on group-by attributes, then scan relation and compute aggregate for each group.
    - Select dept, avg(salary) from emp group by dept;
    - Sort emp table by dept column and then read the sorted tuples in the sequence of dept to compute avg(salary) for each dept.
  - Can improve upon this by combining sorting and aggregate computation.
    - In the last stage of sort, while writing out the sorted tuples, do the computation of avg(salary) for each dept (saves a read of the whole sorted table)



# Aggregate Operations

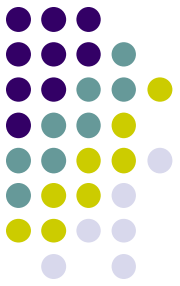


- With grouping:
  - Similar approach based on hashing on group-by attributes.
    - Hash the tuples of emp on dept values.
    - Then compute avg(salary) for each dept by going through the buckets of the hash table.
  - Given tree index whose search key includes all attributes in SELECT, WHERE and GROUP BY clauses, can do index-only scan.
    - Select avg(salary) from emp where job='LEC' group by dept;
    - If there is an index on <dept, job, salary>, do index only scan.



# Aggregate Operations

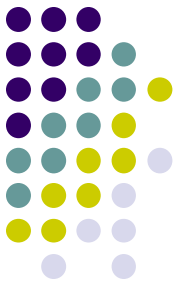
- With grouping:
  - If group-by attributes form prefix of search key, can retrieve **index** entries/tuples in group-by order.
  - **select job, count(\*) from emp group by job;**
    - If the index is on <job, dept, salary>, do an index only scan.
  - **select pno, avg(qty) from sp group by pno;**
    - If the index is on <pno, sno>, then retrieve the sp tuples in pno order using the index.



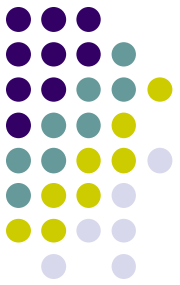
# Choice of low level procedures

- Optimiser chooses one or more candidate procedures for each low level operation
  - using information from catalog about indexes, sizes of relations, etc.
  - this process is sometimes called **access path selection** (this term is used by some to cover both Stage 3 and Stage 4)
    - The procedure is tied up with the access path.
    - For example, separate selection algorithms for file scan, for using primary index, etc.

# Step 4: Generate query plans and choose the cheapest



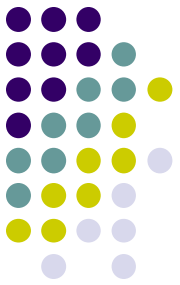
- Construct a set of candidate query plans and choose the best of those plans
- Query plans are built by combining low-level operations in different ways
  - could be combinatorially expensive to do!
  - Many possible procedures for each operation in the query tree.
  - For example, 4 ways each for two selections, 5 ways to join and 3 ways to project gives  $8 \times 5 \times 3 = 120$  ways to execute a simple query.
- Use heuristics to reduce the search space of alternate plans
  - e.g. ignore expensive low-level procedures
- Choose the cheapest plan



# Choosing the cheapest plan

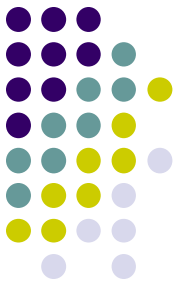
- Requires assigning cost to a plan
- Cost of a plan is the sum of the costs of the individual procedures that make up the plan
  - Use the estimation methods discussed earlier for selection, join, and grouping.
- Cost formulae estimate
  - number of disk I/O's
  - CPU utilisation (we ignore this in our calculations)
- Intermediate results also cause disk I/O's
  - difficult to estimate as they are dependent on actual data values

# Enumeration of Alternative Plans

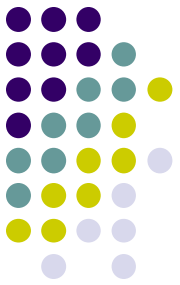


- There are two main cases:
  - Single-relation plans
  - Multiple-relation plans (we consider only 2-relation queries)
- Queries over a single relation consist of a combination of selects, projects, and aggregate ops:
  - Each available access path (file scan / index) is considered, and the one with the least estimated cost is chosen.
  - The different operations are essentially carried out together.
  - e.g., if an index is used for a selection, projection is done for each retrieved tuple, and the resulting tuples are *pipelined* into the aggregate computation.
  - Unless sorting is required for aggregate computation, the query can be executed by reading the tuples at most once.

# Queries Over Multiple Relations



- We consider only two relation queries.
- Enumerated using 2 passes:
  - Pass 1: Find best 1-relation plan for each relation.
  - Pass 2: Find best way to join result of each 1-relation plan (as outer) to the other relation. (All 2-relation plans.)
- Retain only:
  - Cheapest plan overall, plus
  - Cheapest plan for each interesting order of the tuples.
- ORDER BY, GROUP BY, aggregates etc. handled as a final step, using either an 'interestingly ordered' plan or an additional sorting operator.



# Cost of 2-relation Plans

```
SELECT attribute list  
FROM relation1, relations2  
WHERE term1 AND ... AND termk
```

- Consider the above query block.
- Maximum # tuples in result is the product of the cardinalities of relations in the FROM clause.
- *Reduction factor (RF)* associated with each *term* reflects the impact of the *term* in reducing result size.
- *Result cardinality* = Max # tuples \* product of all RF's.
- Cost of join method, plus estimation of join cardinality gives us both cost estimate and result size estimate



# Example

S:

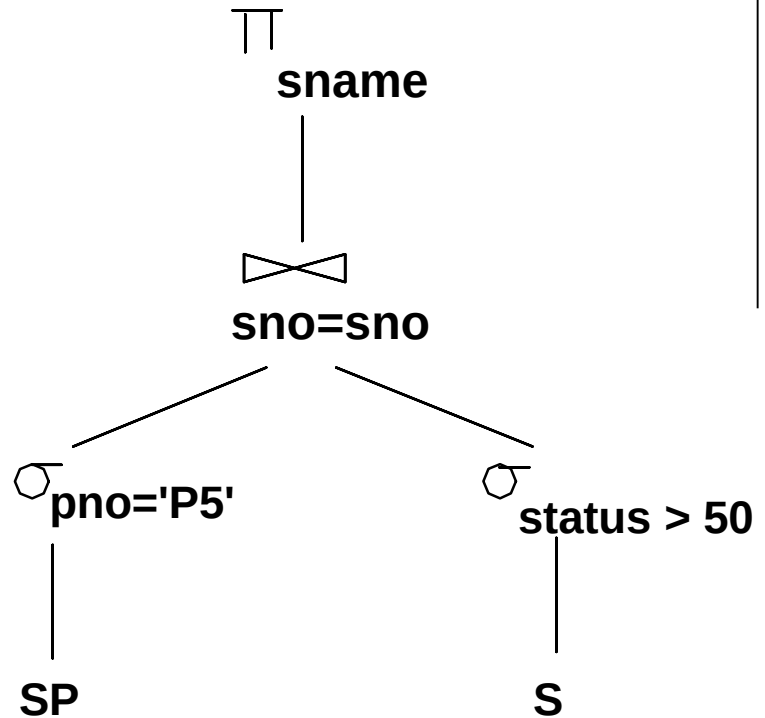
B+ tree on *status*

Hash on *sno*

SP:

B+ tree on *pno*

*All indexes are unclustered.*

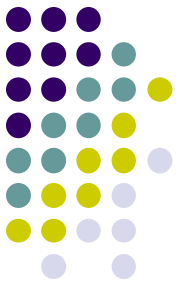


## ● Pass1:

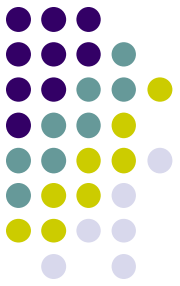
- S: Consider 2 access methods (Full file scan, Index scan with B+ tree)

- Assume min status = 0, max status = 100 and uniform distribution  
status > 50, RF = 50 %
- Full file Scan  
Cost = 100 IOs
- Index Scan with B+ on S<status>

$$Cost = h + RF * (LeafPages + Data Record) = 2 + \frac{50}{100} * (25 + 1000) = 2 + 13 + 500 = 515 \text{ IOs}$$



# Example



- Pass1:
  - SP: Consider 2 access methods (Full file scan, Index scan with B+ tree)
    - pno='P5', RF = 0.5%
    - Full file scan  
Cost = 1000 IOs
    - Index Scan with B+ on SP<pno>

$$Cost = h + RF * (Data\ Pages + Data\ Records)$$

$$= 3 + \frac{0.5}{100} * (334 + 100000)$$

$$= 3 + 2 + 500 = 505\ IOs$$

# Example

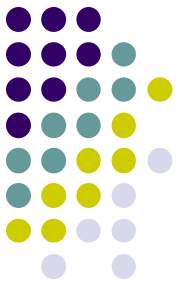
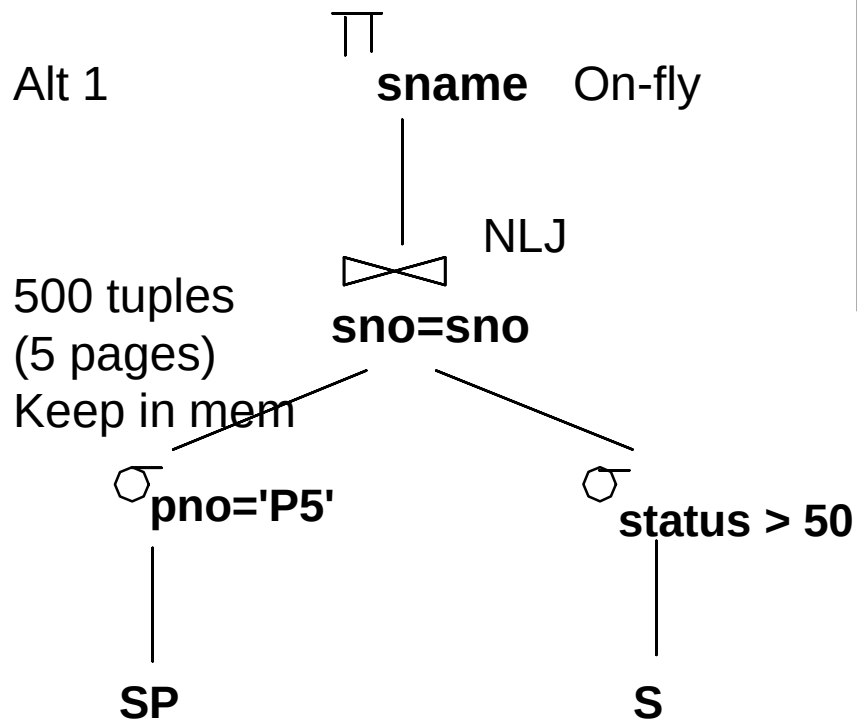
## Pass 2:

- Index scan on SP using B+
- 500 SP tuples satisfy select condition
- Result can be kept in mem.
- Result joined with S using NLJ
- S will be accessed using full file scan
- Join result pipelined to projection to project on fly.

Cost = Cost of  $\sigma$  + cost of join + cost of  $\pi$

$$= 505 + (M + M * N) + 0$$

$$= 505 + (0 + 1 * 100) = 605 \text{ IOs}$$

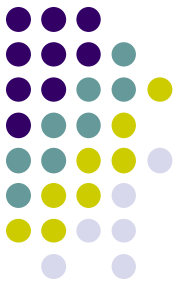


## Alt 2

- Index scan on SP using B+
- 500 SP tuples satisfy select condition

- 
- 500 tuples  
(5 pages)  
Keep in mem
- INLJ
- $\sigma_{sno=sno}$
- $\sigma_{pno='P5'}$
- SP

# Summary



- A query is evaluated by converting it to a tree of algebraic operators.
  - Apply transformation rules to ‘improve’ the query tree.
- Two parts to optimizing a query tree:
  - Consider a set of alternative plans using alternative access paths and algorithms for each operator.
  - Estimate cost of each plan that is considered.
    - Need to estimate size of result and cost for each operation.
    - **Key issues:** Statistics, indexes, operator implementations.